ELECTRÓNICA DIGITAL I · SEGUNDA ENTREGA · CLASE 1 DE 5

Flip-flops, registros y reset

Del sistema combinacional de la entrega 1 a un sistema que recuerda: claves, tiempos y estados.

RUTA DEL CURSO

Cinco clases, un bloque nuevo cada vez

Clase	Tema	Bloques que construyen	Testbench
1 · hoy	**Flip-flops, registros y reset**	**sincronizador, reset_sync, detector_flanco, reg_desplazamiento**	**tb_clase1**
2	Contadores y base de tiempo	base_tiempo, temporizador, antirrebote, temporizador_bcd	tb_clase2
3	Teclado matricial	barrido_teclado, teclado_4x4	tb_teclado
4	Máquinas de estado	fsm_proceso	tb_fsm_proceso
5	Integración y Quartus	fsm_sesion, top_tanque, testbench final	tb_top_tanque

Cada clase deja archivos que se usan en la siguiente. Si un bloque no pasa su testbench, el siguiente no va a funcionar.

REVISIÓN DE LA ENTREGA 1

Cinco cosas que corregir de la entrega 1

Lo que encontramos	Por qué importa	Se corrige en
El .qsf no asigna ningún pin	Quartus los ubicó solo: clk quedó en E1, no en el reloj de 50 MHz (R8)	Clase 5
E/S en 2,5 V por defecto	Los sensores, LEDs y el teclado trabajan a 3,3 V	Clase 5
Decodificador sin el 8; el 9 con el patrón del 8	No se notaba porque solo se mostraban 0, 1, 2, 5 y 7	Clase 5
Registro de salida sin reset	Al encender, las salidas arrancan en un valor desconocido	**Hoy**
priority_coder da 000 para 0 % y para error	La FSM no podría distinguir tanque vacío de sensor dañado	Clase 5

EL SALTO DE ESTA ENTREGA

Una salida secuencial depende de lo que pasó antes

Combinacional · entrega 1

salida = f(entradas de ahora)

Los sensores marcan 0111 → el display dice 75. Si los sensores cambian, el display cambia. No hay nada que recordar.

Secuencial · entrega 2

salida = f(entradas, estado guardado)

La compuerta sigue abierta en 50 % porque **antes** llegó a 100 %. La tecla 4 significa cosas distintas según el menú en que estemos.

Todo registro del proyecto se escribe así

```
process (clk)
begin
  if rising_edge(clk) then
    if rst = '1' then
      q <= '0';
    elsif ena = '1' then
      q <= d;
    end if;
  end if;
end process;
```

rising_edge(clk)

Solo cambia en el flanco de subida. Esa palabra crea el flip-flop.

rst dentro del flanco

Reset síncrono (R2): tiene prioridad y también espera al reloj.

ena sin else

Si ena = 0, q conserva su valor. Aquí la falta de else es memoria correcta.

EL ERROR MÁS COMÚN · R12

Un if incompleto sin reloj fabrica un latch

MAL · latch inferido

```
process (habilitar, dato)
begin
  if habilitar = '1' then
    q <= dato;
  end if;
end process;
```

Si habilitar = 0 no se dice cuánto vale q, así que debe recordar. Sin reloj, eso es un latch.

BIEN · valor por defecto

```
process (habilitar, dato)
begin
  q <= '0';
  if habilitar = '1' then
    q <= dato;
  end if;
end process;
```

Toda rama asigna q. Quartus genera solo compuertas.

Las señales se actualizan al final del process

```
if rising_edge(clk) then
  a <= d;
  b <= a;
  q <= b;
end if;
```

Señales: cada una lee el valor anterior de la otra. Resultado: **3 flip-flops** en cadena. El orden de las líneas no importa.

```
if rising_edge(clk) then
  va := d;
  vb := va;
  q  <= vb;
end if;
```

Variables: cambian al instante. El dato llega a q en un ciclo: **1 flip-flop**. El orden sí importa.

Por eso R8 prohíbe variable en código sintetizable: todo lo que se recuerda debe ser una signal visible.

RESET · R2

Un solo reset, síncrono y limpio



Por qué 2 FF

El botón se suelta en cualquier instante. Sin sincronizar, unos bloques salen de reset un ciclo antes que otros.

Por qué invertir

Dentro del diseño todos usan rst = '1'. Se invierte una sola vez, aquí.

Pregunta

Los FF arrancan en 0 al programar la FPGA. ¿Qué vale rst en los primeros ciclos? ¿Es bueno?

METAESTABILIDAD · R11

Ninguna entrada del mundo real conoce nuestro reloj

Si el teclado o un sensor cambia justo en el flanco del reloj, el flip-flop puede quedar un instante en un valor intermedio, ni 0 ni 1.

Con un segundo flip-flop, el primero tiene un ciclo completo (20 ns) para decidirse antes de que alguien lo lea.

Qué se sincroniza

Sensores S1 a S5 · 5 bits

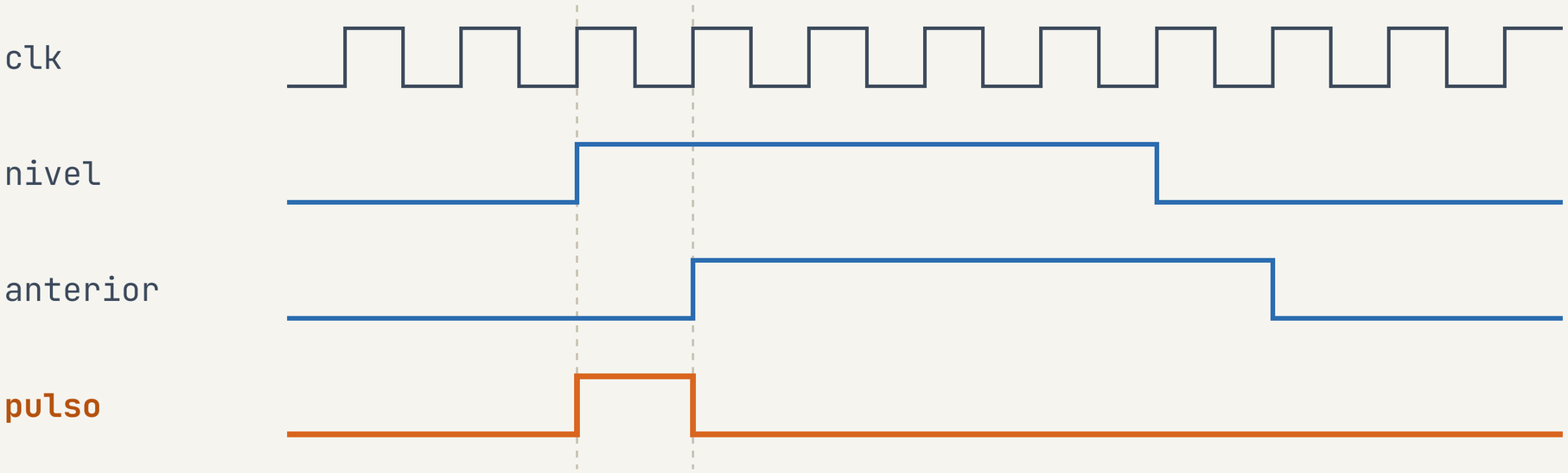
Columnas del teclado · 4 bits

Pulsador de reset · 1 bit

Un solo bloque genérico en ANCHO sirve para los tres.

DETECTOR DE FLANCO

Una tecla mantenida debe contar una sola vez



anterior = nivel retrasado un ciclo (un flip-flop) · **pulso** = nivel AND NOT anterior

REGISTRO DE DESPLAZAMIENTO · R9

La contraseña entra dígito a dígito por la derecha

Operación	q (4 dígitos)	n	Display
clr	0 0 0 0	0	b
push 1	0 0 0 1	1	b -
push 2	0 0 1 2	2	b --
push 3, 4	1 2 3 4	4	----
push 9	1 2 3 4	4	ignorado
pop (*)	0 1 2 3	3	b ---

push: todo se corre un dígito a la izquierda y entra el nuevo.

pop: se corre a la derecha y entra 0000.

Al presionar # se comparan los 16 bits de una vez. Nunca dígito por dígito.

TALLER EN CLASE · 45 MINUTOS

Completen los TODO hasta ver 13 OK

1 · sincronizador.vhd

Dos flip-flops en cascada. Salida = ff2.

2 · reset_sync.vhd

El mismo sincronizador, más la inversión.

3 · detector_flanco.vhd

Un flip-flop y la ecuación de la diapositiva 10.

4 · reg_desplazamiento.vhd

push y pop con concatenación &.

Simulen tb/tb_clase1.vhd con run -all. Al final debe aparecer **RESUMEN: 13 OK, 0 FALLAS**.

PARA LA PRÓXIMA CLASE

Lleguen con los cuatro bloques en verde

- tb_clase1 con 13 OK y 0 FALLAS. Guarden la captura del resumen.
- Lean las secciones 7 y 9 de la guía: base de tiempo y restricciones R1 a R13.
- Piensen: ¿cuántos ciclos de un reloj de 50 MHz caben en un segundo? ¿Cuántos bits necesita un contador para llegar ahí?